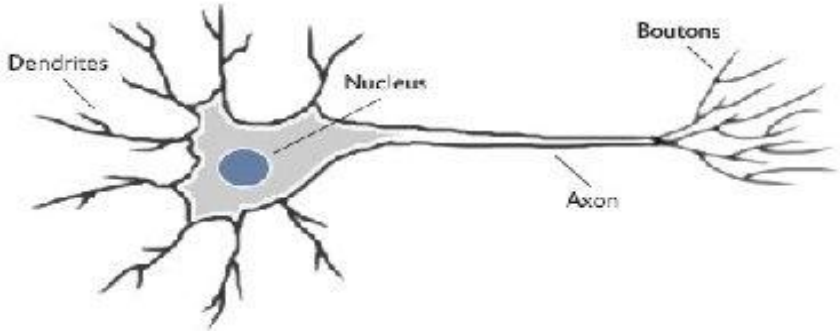
Lecture 5

Introduction to Neural Networks

Motivation

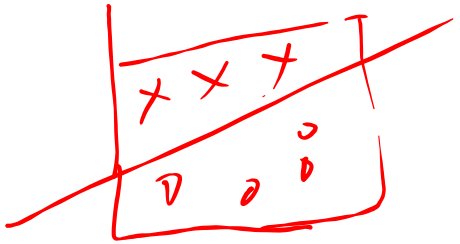
MOTIVATION

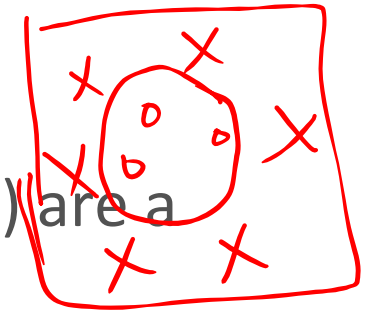
- Our brain uses the extremely large interconnected network of neurons for information processing and to model the world around us. Simply put, a neuron collects inputs from other neurons using *dendrites*. The neuron sums all the inputs and if the resulting value is greater than a threshold, it fires. The fired signal is then sent to other connected neurons through the axon.



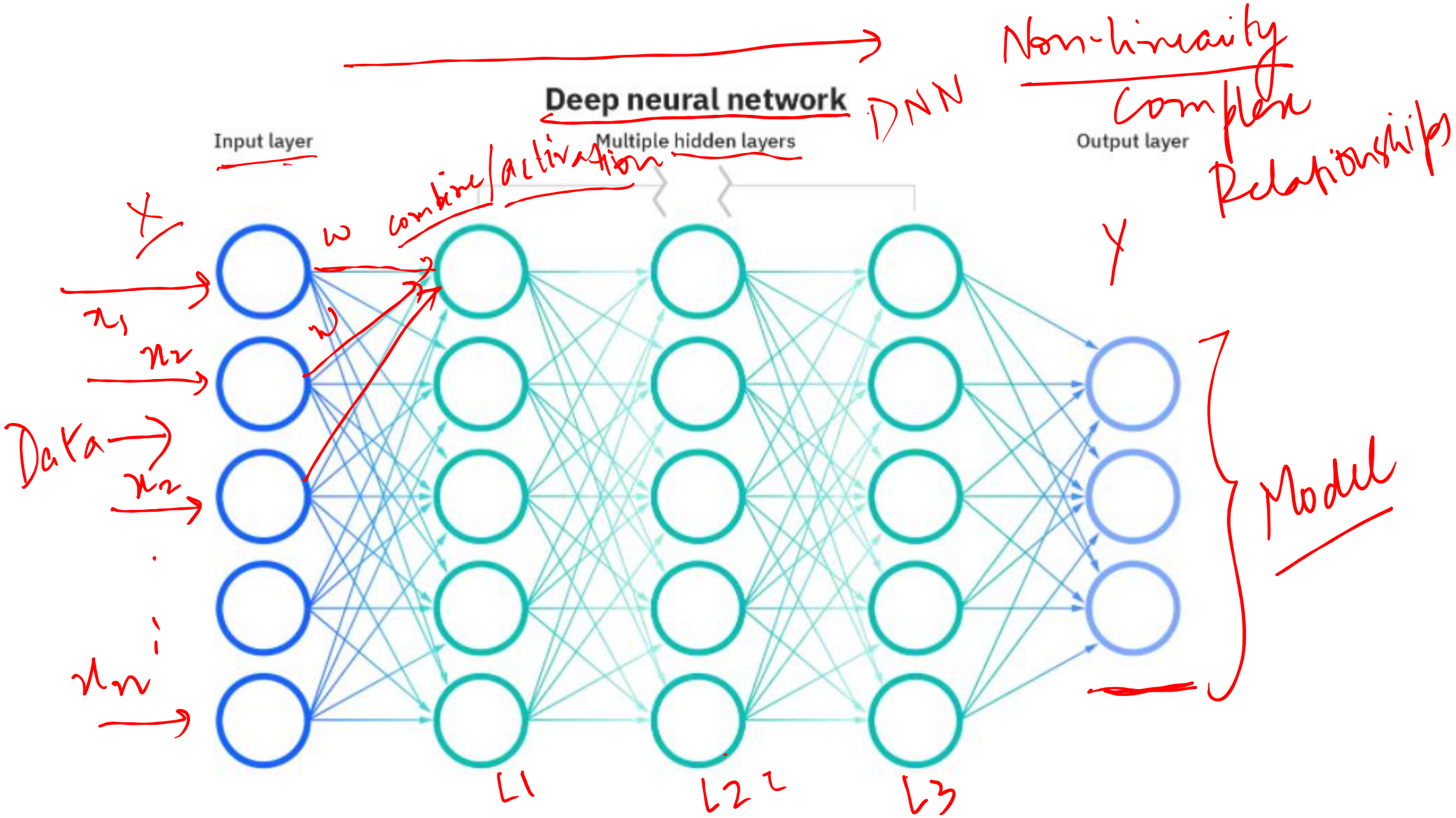
Source: https://medium.com/technologymadeeasy/for-dummies-the-introduction-to-neural-networks-we-all-need-c50f6012d5eb

What are Neural Networks?





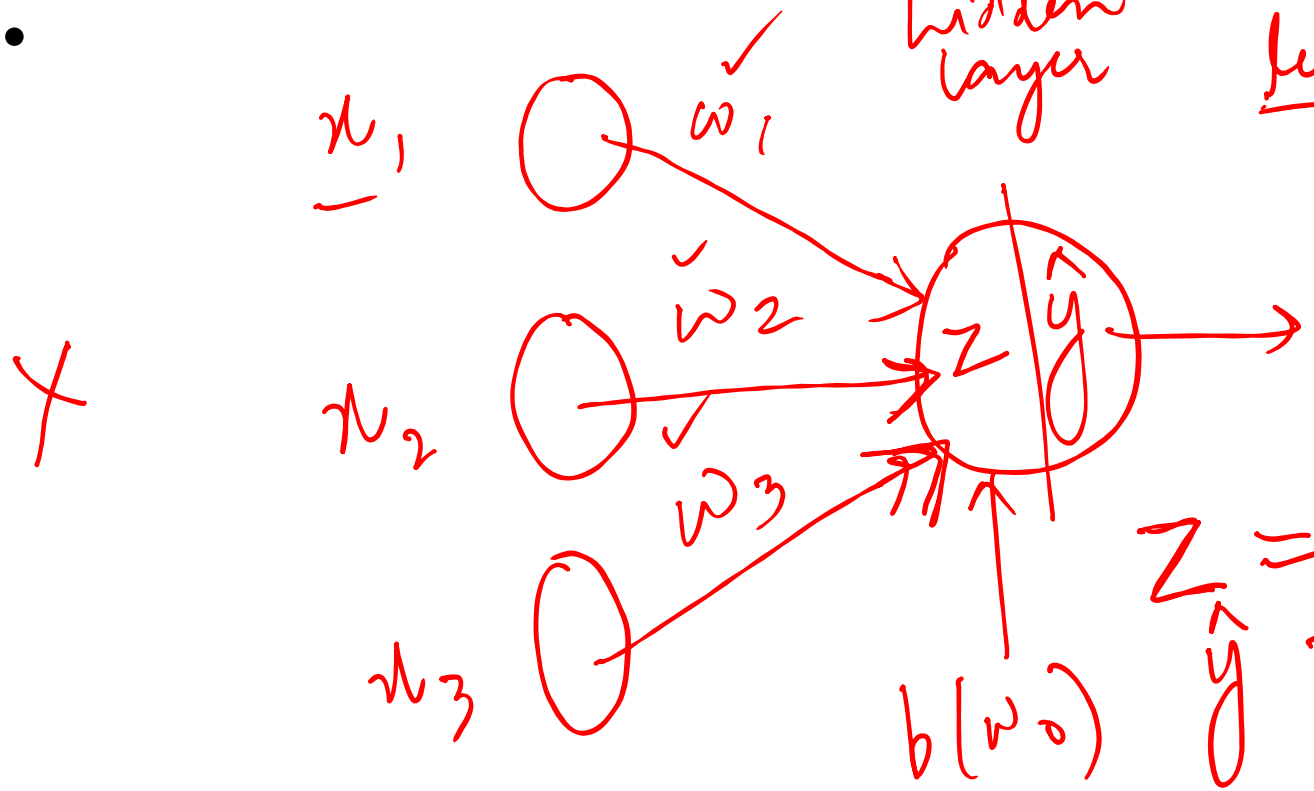
- Neural networks, also known as artificial neural networks (ANNs) are a subset of machine learning and are at the heart of deep learning algorithms.
- Their name and structure are inspired by the human brain, mimicking the way that biological neurons signal to one another.
- Artificial neural networks (ANNs) are comprised of a node layers, containing an input layer, one or more hidden layers, and an output layer.
- Each node, or artificial neuron, connects to another and has an associated weight and threshold.
- The weights and threshold decides what information can be passed to the next layer.
- Source: ibm.com


Deep neural network
DNN
Non-linearity
complex
Relationships
Input layer
Multiple hidden layers
Output layer
X
w
combine/activation
Y
x_1
x_2
Data
x_2
x_n
L1
L2
L3
Model

Computational Graphs

Logistic Regression

-



left to right (one direction)

$$z = w_1x_1 + w_2x_2 + w_3x_3 + w_0$$

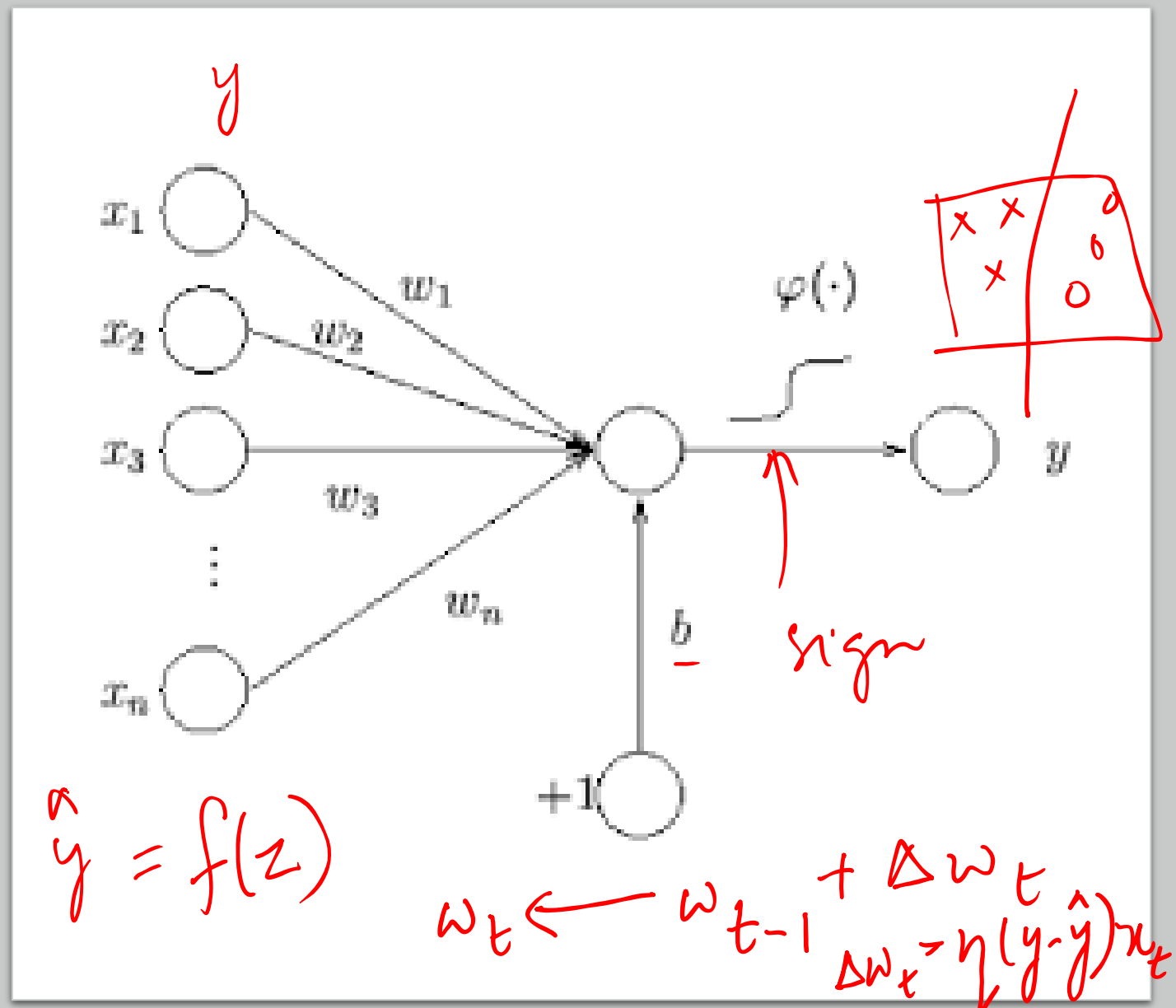
$$\hat{y} = \sigma(z) \rightarrow \text{Activation}$$

$$L(\hat{y}, y)$$

The Perceptron

non-linear data

- The simplest neural network is referred to as a perceptron.
- This comprises of single input layer and an output layer.
- The predicted outcome is passed through a sign function for a binary classification task.
- The weights are updated in the negative direction of the error gradient (gradient descent).



$\hat{y} = f(z)$

$w_t \leftarrow w_{t-1} + \Delta w_t$

$\Delta w_t = \eta(y - \hat{y})x_t$

Representation

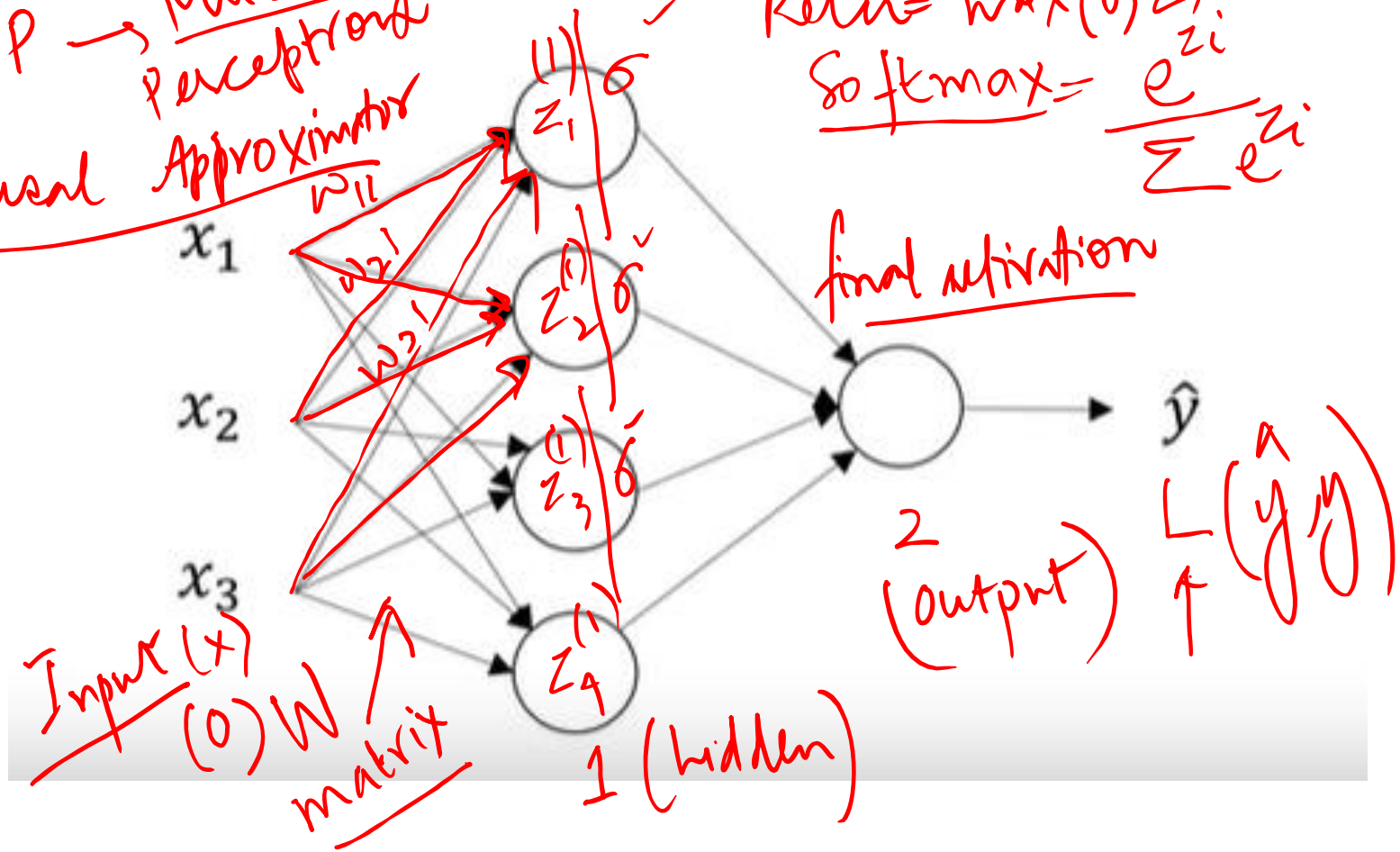
MLP → Multilayered Perceptron

Universal Approximator

Activation

Relu = max(0, z)

Softmax = $\frac{e^{z_i}}{\sum e^{z_i}}$



Compute NN Output

- Forward Propagation

1) Pass X (data) into the network (Input layer)

2) Hidden layer computation (l = layer index)

$$Z^{(l)} = W^{l} \cdot a^{(l-1)} + b^{(l)}$$

where $a^{(l)}$ is activation $\rightarrow a^{(l)} = f(Z^{(l)})$

3) Output layer: Regression $\hat{y} = Z^{(L)}$

classification $\hat{y} = \text{softmax}(Z^{(L)})$

$\text{sigmoid}(Z^{(L)})$

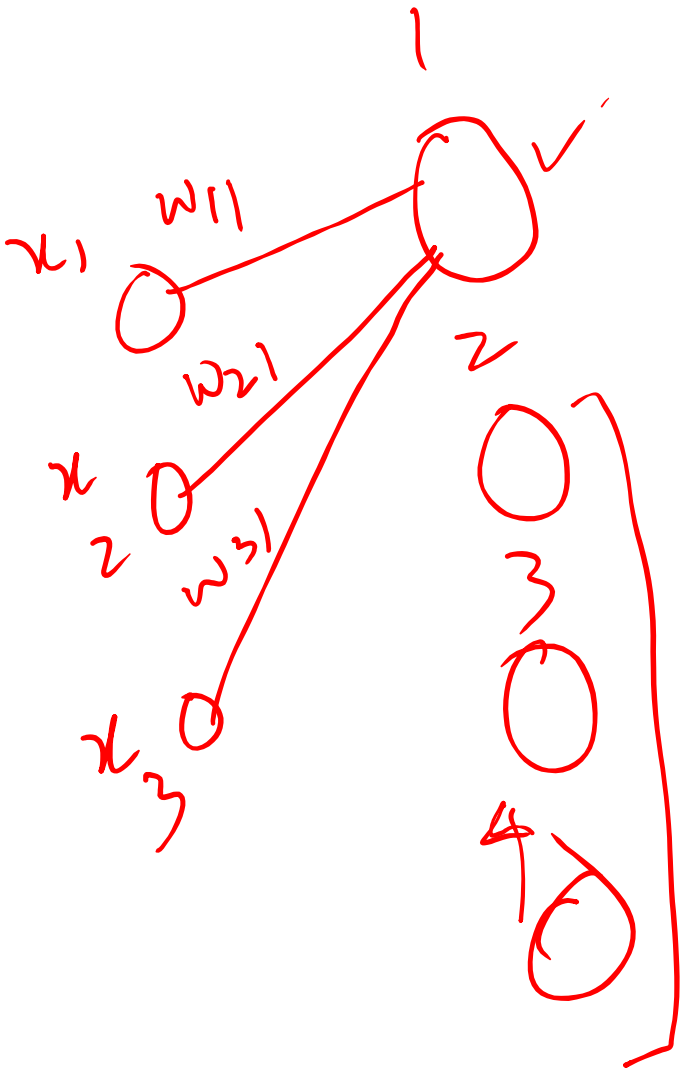
$W^{[l]} X \longleftrightarrow XW$

matrix

vector

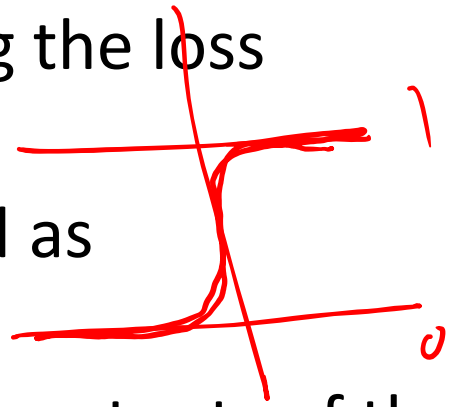
Dimensionality

$$\begin{bmatrix} w_{11} & w_{21} & w_{31} \\ w_{12} & w_{22} & w_{32} \\ & & \end{bmatrix}_{4\times 3} \begin{bmatrix} x \end{bmatrix}_{3\times 1}$$



Activation Functions

- Sign: Non-differentiability prevents its use for creating the loss function at training time.
- Sigmoid: Useful for computations that are interpreted as probabilities.
- Hyperbolic tangent : Preferred over sigmoid when the outputs of the computation are desired to be both positive and negative. Due to mean-centering and larger gradient its easier to train.



Stronger gradients
faster learning

$$\tanh = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}} \rightarrow -1 \times 1$$

Piecewise linear Activation Functions

These have become popular due to ease of training with multi-layered neural networks:

- Rectified Linear Unit (ReLU) $= \max(0, z) = \begin{cases} z & \text{if } z > 0 \\ 0 & \text{if } z \leq 0 \end{cases}$
- Hard tanh

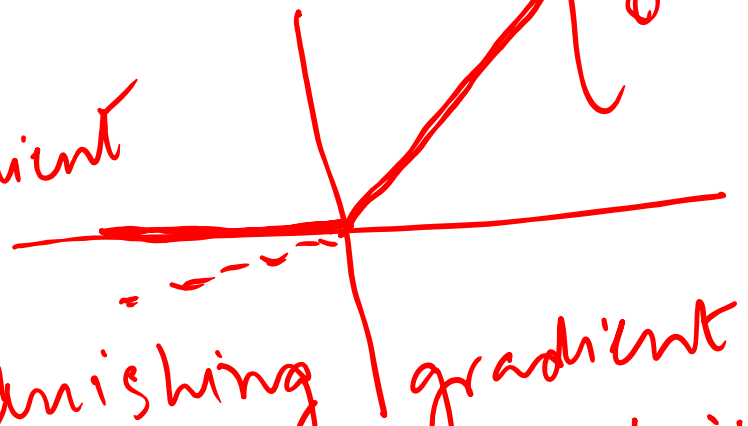
0.0001×0.0002

→ small

↳ set it to zero

1) Sparse activation.

2) Better gradient propagation



→ Vanishing gradient problem.

→ Leaky Relu, Parametric Relu

Gradient Descent for Neural Networks

- To find optimal weights we use gradient descent.

1) Forward Propagation
   - Initialize weights → random
   - Compute output at each layer
   - Cache intermediate results, $z^{(l)}$, $a^{(l)}$, $W^{(l)}$
2) Make initial prediction
   - Loss $= L(y, \hat{y})$: optimize (GD)
3) Backpropagation Algorithm calculate gradient of loss w.r.t. weights

Random initialization

- $W^{[1]} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$ $\quad b^{[1]} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$

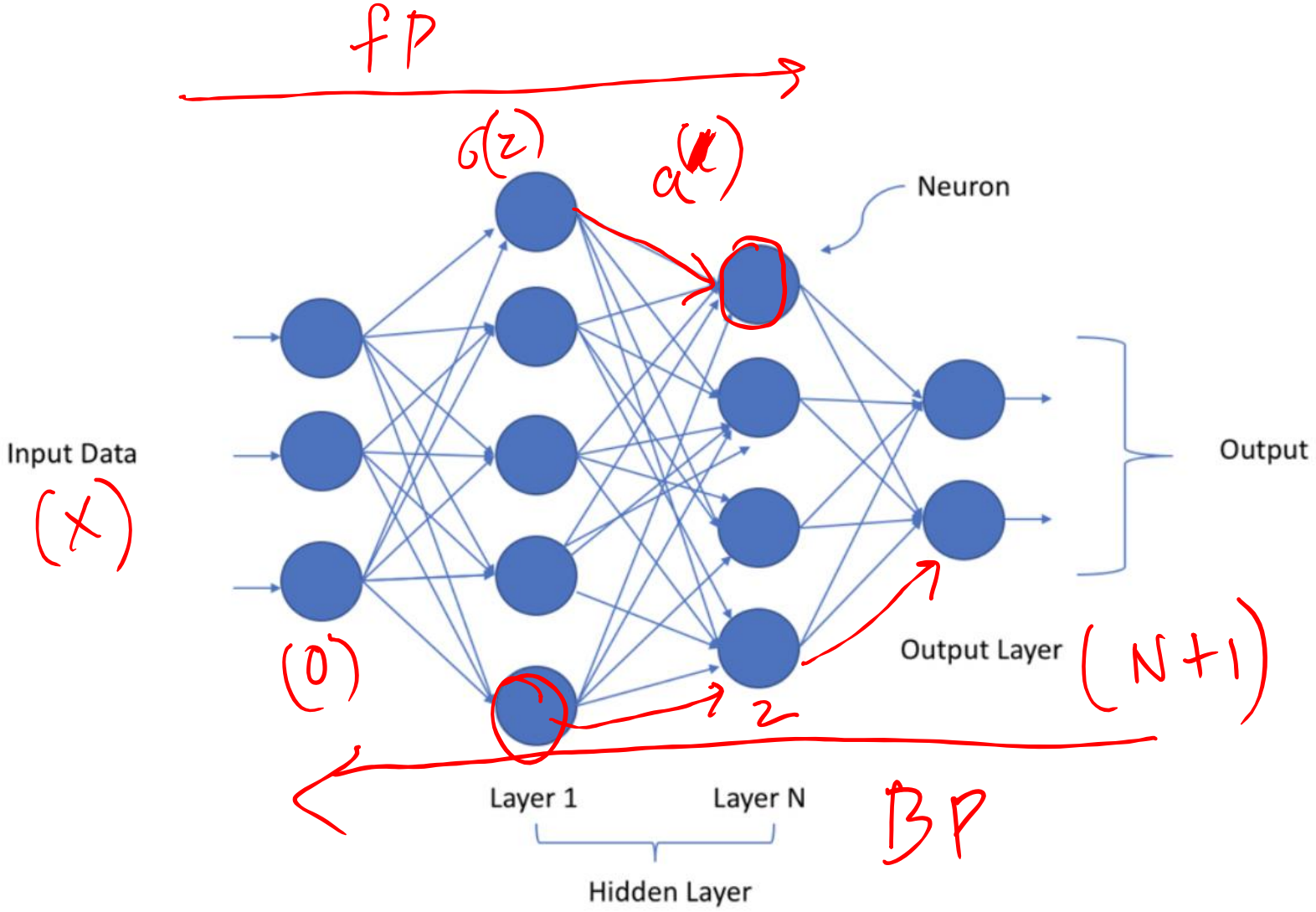
symmetry gradient problem

$a_1^{[1]} = a_2^{[1]}$ $\quad dz_1^{[1]} = dz_2^{[1]}$

$\frac{dL}{dz}$

$$W^{[1]} = W^{[1]} - \eta \frac{dL}{dW}$$

Deep Neural Networks



Forward Propagation

- Computations done in the graph
- Applying summations and activations to calculate the prediction.

1) First Layer: Input $X = [x_1, x_2, x_3]$

$W^{(1)}_{4\times3}$, $b^{(1)}_{4\times1}$, $Z^{(1)} = W^{(1)} X^T + b^{(1)}$

$a^{(1)} = \sigma(Z^{(1)})$ $\left\{ \frac{1}{1+e^{-z}} \right.$

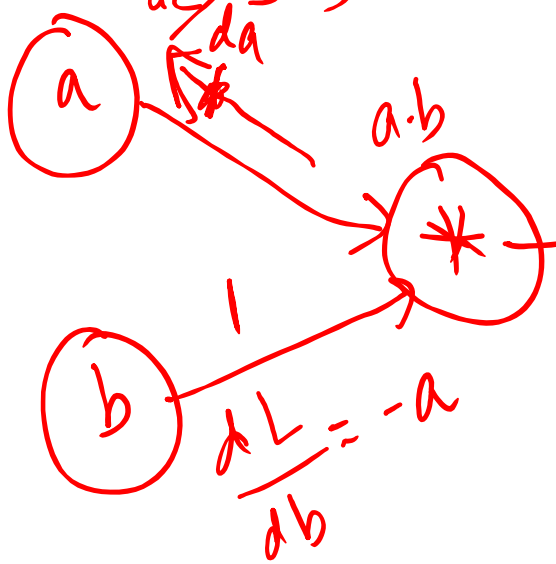
2) Second layer: Input $= a^{(1)}$

$W^{(2)}_{\underline{2\times4}}$ $b^{(2)}_{\underline{2\times1}}$ $Z^{(2)} = W^{(2)} a^{(1)} + b^{(2)}$

$a^{(2)} = f(Z^{(2)})$

Backpropagation

- Intuition → Multiplication



$a = a - \eta \frac{dL}{da}$

→ $L = y - \hat{y}$

$\hat{y} = a.b$

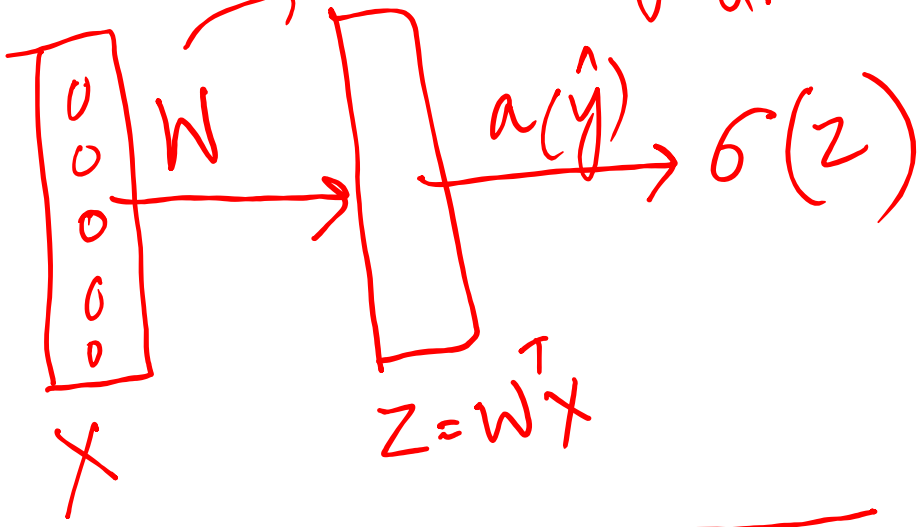
$\frac{dL}{d\hat{y}} = -1$

$\frac{dL}{da} = \frac{dL}{d\hat{y}} \frac{d\hat{y}}{da} = -b$

$\frac{dL}{db} = \frac{dL}{d\hat{y}} \cdot \frac{d\hat{y}}{db} = -a$

Logistic Regression

$$W = W - \eta \frac{dL}{dW}$$



Classification

$$L(a, y) = -y \log a - (1-y) \log(1-a)$$

1) $$\frac{dL}{da} = -\frac{y}{a} - \frac{(1-y)}{1-a}$$ ✓

$$\frac{da}{dz} = a(1-a)$$ ✓ $\left[\sigma' = \sigma(1-\sigma)\right]$

Goal

$$\frac{dL}{dW} = \frac{dL}{dz} \cdot \frac{dz}{dW}$$

$$= (a-y)X$$

$$\left(\frac{dz}{dW}\right)^* = X, \quad \frac{dL}{dz}^* = \frac{dL}{da} \cdot \frac{da}{dz}$$

$$= (a-y)$$

Generalize Results

$$\frac{dL}{dW}(dW), \quad dW^{(2)} = \left(A^{(2)} - Y^{(2)}\right) X$$

for other layers

$$\underline{dW^{[1]}} = dZ^{[2]} \cdot X$$

$X \rightarrow$ input from previous layer

$X \rightarrow A^0 = X$

$$dW^{(2)} = \cancel{dZ^{(2)} A^{(1)}}$$

$$dW^{(2)} = dZ^{(2)} A^{(1)}$$

$$db^{(2)} = dZ^{(2)}$$

$$dW^{(1)} = dZ^{(1)} \cdot X$$

$$\left(\frac{dL}{dZ}\right) \quad \underline{\underline{dZ^{(1)}}} = W^{2}\, dZ^{(2)} * \sigma'(Z^{(1)}) \longrightarrow \left\{ da \cdot \frac{da}{dz} \right.$$

logistic reg.

elementwise

$$db^{(1)} = dZ^{(1)}$$

Summarize BP → 4 equations

$$dz^{(l)} = da^{(l)} * g'(z^{(l)})$$

$$dW^{(l)} = dz^{(l)} \, a^{(l-1)}$$

$$db^{(l)} = dz^{(l)}$$

$$da^{(l)} = W^{(l+1)} \cdot dz^{(l+1)}$$

$$W^{l} = W^{(l)} - \eta \, dW^{(l)}$$

update step

The Backpropagation Algorithm Summarized.

- Perform forward propagation.
- Compute the partial derivative for the output layer
- Compute the partial derivatives of the cost function with respect to all of the parameters that feed into the output layer
- Go back one layer
- Compute the partial derivative of the output term for the current hidden layer
- Compute the partial derivatives of the cost function with respect to all of the parameters that feed into the current layer.
- Repeat steps 4 through 6 until you reach the input layer.

References

https://www.jeremyjordan.me/neural-networks-training/